



南开大学  
Nankai University

# 《软件文化概论》课程报告

## 我所学到的软件文化

学    院： 软件学院

姓    名： 陈宇翔

学    号： 2312887

2024~2025 第一学期

使用 Office Word 撰写于 2024 年 12 月 29 日

## 目 录

|     |                 |   |
|-----|-----------------|---|
| 1   | 课程内容的扩展 .....   | 2 |
| 1.1 | 深度学习的发展历程 ..... | 2 |
| 1.2 | 算法简析 .....      | 3 |
| 1.3 | 数据库简介 .....     | 3 |
| 2   | 课程的收获与改进 .....  | 4 |
| 2.1 | 课程收获 .....      | 4 |
| 2.2 | 改进意见 .....      | 4 |
| 3   | 结语 .....        | 4 |
|     | 参考文献 .....      | 5 |

## 1 课程内容的扩展

通过本课程学习，我了解到了许多与软件文化有关的知识。在一堂堂生动而不失专业性的讲座上，老师们为我们讲述了与操作系统、数据库、人工智能、大模型、智能运维和物联网等有关的内容。当然，课堂的时间有限，为此我在课下的时间选取了以下几个感兴趣的方面去查询相关资料以深入了解。

### 1.1 深度学习的发展历程

在各位老师的讲座当中，AI一词被提及的频率是相当高的。而人工智能的崛起，离不开深度学习的发展。

深度学习的起源最早可追溯至1943年，在这一年神经学家沃伦·麦卡洛克与数学家沃尔特·皮茨共同提出了M-P模型，这是最早的神经网络模型，它是基于生物神经元的结构和功能进行建模的。<sup>【1】</sup>通过分析神经递质在神经元间的传输方式，他们得出了神经元的输出模型：

$$y = f\left(\sum_1^n w_i x_i - \theta\right)$$

其中  $\theta$  为神经元的激活阈值，函数  $f(x)$  是被激活函数，通常用Sigmoid函数表示，这便是高中生物中的S型增长曲线，常用于表示大部分种群在有限资源下的实际增长速率。而在信息科学中，由于其单增及反函数单增等性质，Sigmoid函数常被用做神经网络的激活函数，将变量映射到0-1之间。即

$$f(x) = \frac{1}{1 + e^{-ax}} \quad (0 < f(x) < 1)$$

六年之后的1949年，加拿大生理心理学家唐纳德·赫布出版了《行为的组织》一书，在其中提出了Hebb学习规则（又称相关学习或并联学习）。<sup>【1】</sup>这是一个无监督学习规则，它只根据神经元连接间的激活水平改变权值，因此描述了神经元之间连接强度的变化规律。<sup>【2】</sup>

1957年，美国心理学家和计算机科学家弗兰克·罗森布拉特提出了单层感知器模型，这是最早的前馈神经网络之一，它标志着深度学习的发展进入了感知器时代。<sup>【1】</sup><sup>【3】</sup>作为一种简单的神经网络结构，它主要用于解决二分类问题，通过特征向量把矩阵上的输入映射到输出值  $f(x)$  上。

$$f(x) = \begin{cases} 1, & \text{if } w \cdot x + b > 0 \\ 0, & \text{else} \end{cases}$$

$w$ 是实数的表式权重的向量， $w \cdot x$ 是点积。 $b$ 是偏置，一个不依赖于任何输入值的常数。偏置可以认为是激励函数的偏移量，或者给神经元一个基础活跃等级。<sup>【4】</sup>感知器模型在当时被认为有着良好的发展潜能。

在1986年，David Rumelhart、Geoffrey Hinton和Ron Williams三人提出了反向传播算法，也就是大家所熟知的BP算法。<sup>【3】</sup>它建立在梯度下降法的基础上，其信息处理能力来源于简单非线性函数的复合，因此具有很强的非线性映射能力。<sup>【2】</sup>BP算法通过调整权重来最小化输出误差，它的出现使人们能有效训练多层神经网络。随后在1989年，卷积神经网络（CNN）的出现，大大提升了对图像等高维数据的处理效率。<sup>【5】</sup>

步入21世纪，互联网的发展使得大量数据可以存储在云端，同时图形处理器（GPU）的进步为大规模并行计算奠定了基础，深度学习和神经网络得以井喷式发展。<sup>【3】</sup>2006年，亨特等人提出了“深度信念网络”的概念，解决了深层网络训练中的梯度消失问

题，这开启了深度学习的新纪元。<sup>【6】</sup>而后在2012年，AlexNet——一种深度卷积网络的出现，极大程度上提高了图像分类准确率，引发了深度学习领域的革命。<sup>【7】</sup>在深度学习的研究热潮中，基于缩放定理的大模型脱颖而出，人工智能进入了大模型时代。大模型是指由包含数百亿以上参数的深度学习神经网络构建的语言模型，随着模型参数和预训练数据规模的不断增加，模型的能力与任务效果会持续提升。在这其中最出名、最有影响力的模型基座便为Transformer和Diffusion Model，基于Transformer的ChatGPT就是代表之一。作为一种架构庞大、计算资源密集、特征提取能力强、智能化程度高的机器学习方式，如今深度学习已经广泛运用于各个领域，它极大程度上改变了我们的生活方式。

## 1.2 算法简析

算法是一组严谨地定义运算顺序的规则，并且每一个规则都是有效的，且是明确的，此顺序将在有限的次数下终止。简单的说，算法就是解决问题的手段，并且是批量化解决问题的手段。在数学和计算机科学中，算法是如何解决一类问题的明确规范。算法可以执行计算、数据处理和自动推理任务。<sup>【8】</sup>

**算法的特性：**有穷性、确定性、可行性、零个或多个输入、一个或多个输出。

**算法的表现形式：**自然语言、伪代码、流程图。

**算法的设计要求：**正确性、可读性、高效率与低存储要求。

**算法的分类：**

按目的：数据处理算法、优化算法、加密算法等。

按执行方法：递归算法、递推算法、迭代算法等。

按应用领域：图形算法、网络算法、人工智能算法等。

**算法的应用范围：**排序、搜索、加密、树和图等。

总而言之，算法是解决问题的基础，无论是简单的数据处理还是复杂的机器学习模型，算法都是实现目标的蓝图。<sup>【9】</sup>它通过优化过程、减少时间和空间消耗来提高效率，也能通过重复和验证来确保结果的准确性和可靠性。算法是计算机的灵魂，它引领着计算机技术的发展进步。

## 1.3 数据库简介

数据库（Database）是一个系统化存储、管理和操作数据的集合。它是一个按照特定结构组织的数据集合，可以支持数据的存储、查询、更新和管理。数据库通常包括数据表、数据记录以及相关的操作工具，能够帮助用户高效地进行数据存储、查询与操作。<sup>【10】</sup>

数据库有如下几个基本概念。数据表作为数据库中的基本存储单位，包含数据行（记录）和列（字段）。数据行是数据表中的一行，包含一个实体或对象的所有信息。数据列是数据表中的一列，表示某个具体的属性或数据项。主键是每个记录的唯一标识符，它确保数据的唯一性。而外键通常指向另一张表的主键，用于在不同的数据表之间建立联系。

除了基本概念外，数据库也有着四种典型操作（这些操作与我们在数据结构中所学的搜索树的操作十分相似）。增（Insert）：将数据插入到数据库表中；查（Select）：查询数据库中的数据；改（Update）：修改数据库中的现有数据；删（Delete）：删除数据库中的数据。

数据库有着数据共享和集中管理、数据独立性、一致性、完整性、数据安全性和权限控制、高效的访问和查询速度、可扩展性和可伸缩性等特点。它的分类标准很多，常见的一种便是将其分为关系型数据库和非关系型数据库两类。对于关系型数据库，其数据存储存储在表格中，并通过SQL进行操作；而非关系型数据库的数据则以键值对、文档、图等格式存储，可以支持半结构化和非结构化数据，适合处理大规模、分布式的数据。<sup>[11]</sup>它在企业业务数据管理、金融交易处理、医疗保健管理和教育管理等领域都有着广泛的应用。

## 2 课程的收获与改进

### 2.1 课程收获

- 了解了与软件行业有关的各领域的基础知识。
- 学会了如何借助互联网去深入扩展探索课上所学的内容。
- 学习了如何借助AI大模型以便利自己的日常生活。

### 2.2 改进建议

虽然在提出互动加分的机制后，课堂提问的积极性不断升高，但我认为有些提问带有一定的功利性，且提问环节十分混乱。希望能够建立健全提问互动机制，通过规范的管理让问题整齐化和多样化；同时开设线上提问渠道，能够让像我这样的“i人”也能有更多提问的机会。

## 3 结语

总体来说，软件文化概论是我在本学期所学的一堂非常丰富的课程。通过授课与讲座交替进行的新颖方式，我不仅在其中收获了大量的知识，更重要的是对软件的兴趣和热情被极大程度上的激发，我想这对我们这些刚刚踏入软件学院的本科生来说是十分难

得和可贵的，它激励着我们在学习软件的道路上砥砺前行。在此郑重地向姜桂飞老师以及其他老师表示诚挚感谢，同时也希望这门课程能在今后越办越好，吸引一批又一批的软件人。

## 参 考 文 献

- [1] leah126, 《一文说尽深度学习发展历程！零基础入门到精通，收藏这篇就够了》CSDN, , [一文说尽深度学习发展历程！零基础入门到精通，收藏这篇就够了\\_深度学习原理\\_典型模型发展-CSDN 博客](#)
- [2] 《神经网络编程入门》，[神经网络编程入门 - 苍梧 - 博客园](#)
- [3] 何遇mirror, 《深度学习概览：历史、原理与现代应用》，CSDN, [深度学习概览：历史、原理与现代应用-CSDN博客](#)
- [4] 焦李成, 《神经网络系统理论》，西安：西安电子科技大学出版社，1989
- [5] 邱锡鹏, 《神经网络与深度学习》，北京：机械工业出版社，2020
- [6] 张钹, 朱军, 苏航, 《迈向第三代人工智能》，中国科学：信息科学，2020，50（9）：1281 - 1302
- [7] 《深度学习基础知识：介绍深度学习的发展历程、基本概念和主要应用》，阿里云, [深度学习基础知识：介绍深度学习的发展历程、基本概念和主要应用-阿里云开发者社区](#)
- [8] [什么是算法？-腾讯云开发者社区-腾讯云](#)
- [9] WA-自动机, 《算法：计算机科学的灵魂》，CSDN, [算法：计算机科学的灵魂\\_算法为什么是计算的灵魂-CSDN博客](#)
- [10] 玖玖爱编程, 《了解什么是数据库》，CSDN, [了解什么是数据库（简介）-CSDN博客](#)
- [11] 九章云极DataCanvas, 《什么是数据库》，CSDN, [什么是数据库？数据库有哪些基本分类和主要特点？-CSDN博客](#)

## 附录：关于使用C++实现五子棋的人机对战

对于实现电脑下五子棋，关键在于对棋盘上每个点进行评分。评分越高的点，意味着当前一步下在此处对棋手的收益越大。可以发现，五子棋一般都是从中间开始下棋，随后逐渐向外围扩展。由此可以对棋盘的初始位置赋上初值，使得中间位置比外周的分数要高。如对15\*15的棋盘：

```
const int scoreMap[15][15] =
{
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
    {0,1,1,1,1,1,1,1,1,1,1,1,1,1,0},
    {0,1,2,2,2,2,2,2,2,2,2,2,2,1,0},
    {0,1,2,3,3,3,3,3,3,3,3,3,2,1,0},
    {0,1,2,3,4,4,4,4,4,4,4,3,2,1,0},
    {0,1,2,3,4,5,5,5,5,5,4,3,2,1,0},
    {0,1,2,3,4,5,6,6,6,5,4,3,2,1,0},
    {0,1,2,3,4,5,6,7,6,5,4,3,2,1,0},
    {0,1,2,3,4,5,6,6,6,5,4,3,2,1,0},
    {0,1,2,3,4,5,5,5,5,5,4,3,2,1,0},
    {0,1,2,3,4,4,4,4,4,4,4,3,2,1,0},
    {0,1,2,3,3,3,3,3,3,3,3,2,1,0},
    {0,1,2,2,2,2,2,2,2,2,2,2,1,0},
    {0,1,1,1,1,1,1,1,1,1,1,1,1,0},
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
};
```

而后，对于每个还未落子的点，我们假设在此处落子，对于棋盘上形成的局势进行算分，为此需要一个评估分数表。

评估分数表

| 棋型 | 连五     | 活四    | 冲四   | 活三  | 眠三 | 活二 | 眠二 |
|----|--------|-------|------|-----|----|----|----|
| 分数 | 100000 | 20000 | 2000 | 200 | 75 | 40 | 15 |

```
int MsgProcess::scoretable(int number, int empty1, bool ai)//计分板
{
    if (number >= 5)        return (ai ? 50000 : 100000);
    else if (number == 4)
    {
        if (empty1 == 2)    return (ai ? 10000 : 20000);
        else if (empty1 == 1) return (ai ? 1500 : 2000);
    }
    else if (number == 3)
    {
        if (empty1 == 2)    return (ai ? 180 : 200);
        else if (empty1 == 1) return (ai ? 70 : 75);
    }
    else if (number == 2)
    {
        if (empty1 == 2)    return 40;
        else if (empty1 == 1) return 15;
    }
    //else if (number == 1 && empty1 == 2)    return 5;
    return 0;
}
```

有了评估分数表后，我们便可以对每个点进行算分。  
以下便是一个通过分数表计算得分的评估函数

```
int MsgProcess::evaluate_value(int Color, bool ai)
{
    int score = 0;
    for (int i = 0; i < 15; i++)
    {
        int number = 0;
        int empty = 0;
        for (int j = 0; j < 15; j++)
        {
            if (cell[i][j] == Color)
            {
                number++;
                if (j == 15 - 1 && number > 1 && empty == 1)
                    score += scoretable(number, empty, ai);
            }
            else if (cell[i][j] == -Color)
            {
                if (number > 1 && empty == 1)
                    score += scoretable(number, empty, ai);
                empty = 0;
                number = 0;
            }
            else
            {
                empty++;
                if (number > 1)
                    score += scoretable(number, empty, ai);
                else if (number == 1 && empty == 2)
                    score += 0;
                empty = 1;
                number = 0;
            }
        }
    }
    for (int j = 0; j < 15; j++)
    {
        int number = 0;
        int empty = 0;
        for (int i = 0; i < 15; i++)
        {
            if (cell[i][j] == Color)
            {
                number++;
                if (i == 15 - 1 && number > 1 && empty == 1)
                    score += scoretable(number, empty, ai);
            }
            else if (cell[i][j] == -Color)
            {
                if (number > 1 && empty == 1)
                    score += scoretable(number, empty, ai);
                empty = 0;
                number = 0;
            }
            else
            {
                empty++;
            }
        }
    }
}
```



```
        if (number > 1)
            score += scoretable(number, empty, ai);
        else if (number == 1 && empty == 2)
            score += 0;
        empty = 1;
        number = 0;
    }
}
for (int i = 0; i < 10; i++)
{
    int number = 0;
    int empty = 0;
    for (int j = 0; j < 15 - i; j++)
    {
        if (cell[j][i + j] == Color)
        {
            number++;
            if (i + j == 14 && number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
        }
        else if (cell[j][i + j] == -Color)
        {
            if (number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
            empty = 0;
            number = 0;
        }
        else
        {
            empty++;
            if (number > 1)
                score += scoretable(number, empty, ai);
            else if (number == 1 && empty == 2)
                score += 0;
            empty = 1;
            number = 0;
        }
    }
}
for (int j = 1; j < 10; j++)
{
    int number = 0;
    int empty = 0;
    for (int i = 0; i < 15 - j; i++)
    {
        if (cell[i + j][i] == Color)
        {
            number++;
            if (i + j == 14 && number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
        }
        else if (cell[i + j][i] == -Color)
        {
            if (number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
```

```
        empty = 0;
        number = 0;
    }
    else
    {
        empty++;
        if (number > 1)
            score += scoretable(number, empty, ai);
        else if (number == 1 && empty == 2)
            score += 0;
        empty = 1;
        number = 0;
    }
}

for (int i = 14; i > 4; i--)
{
    int number = 0;
    int empty = 0;
    for (int j = 0; j <= i; j++)
    {
        if (cell[i - j][j] == Color)
        {
            number++;
            if (j == i && number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
        }
        else if (cell[i - j][j] == -Color)
        {
            if (number > 1 && empty == 1)
                score += scoretable(number, empty, ai);
            empty = 0;
            number = 0;
        }
        else
        {
            empty++;
            if (number > 1)
                score += scoretable(number, empty, ai);
            else if (number == 1 && empty == 2)
                score += 0;
            empty = 1;
            number = 0;
        }
    }
}

for (int j = 1; j < 10; j++)
{
    int number = 0;
    int empty = 0;
    for (int i = 14; i >= j; i--)
    {
        if (cell[i][j + 14 - i] == Color)
        {
            number++;
            if (i == j && number > 1 && empty == 1)
```

```

        score += scoretable(number, empty, ai);
    }
    else if (cell[i][j + 14 - i] == -Color)
    {
        if (number > 1 && empty == 1)
            score += scoretable(number, empty, ai);
        empty = 0;
        number = 0;
    }
    else
    {
        empty++;
        if (number > 1)
            score += scoretable(number, empty, ai);
        else if (number == 1 && empty == 2)
            score += 0;
        empty = 1;
        number = 0;
    }
}
}
return score;
}
int MsgProcess::ai_subtract_human(int row, int col, int aiColor)
{
    //cell[row][col] = Color;
    int x = evaluate_value(aiColor, 1) - evaluate_value(-aiColor);
    //cell[row][col] = 0;
    return x;
}

```

## 极大-极小值搜索

什么是极大极小值搜索呢？简单来说，就类似于棋手下棋时不局限于当前棋局，而是“多考虑几步”。对于仅有一层的算分，我们只是简单的计算了当前的分数，即为简单的进攻，而并未考虑到对对手的防守。要想实现防守的效果，我们就必须要多考虑几步。这里AI要假设对手足够聪明，此时将算分划为极大值层和极小值层。极大值层通常是AI的一方，目标是使得自己的得分最大化；而极小值层通常是对手的一方，目标是使得AI的一方得分最小化。在五子棋的AI中，AI通常作为极大值方，而人类玩家作为极小值方。通过不断模拟可能的棋局变化，极大极小值搜索试图为AI选择最优的下一步棋。

极大极小值搜索的算法流程主要分为以下几点：

**树结构：**极大极小算法将整个棋局的可能变化看作一个树状结构，树的每一层代表一方的回合。根节点代表当前棋局的状态，接下来的节点表示不同的可能状态，直到游戏结束（胜利、平局或失败）。

**递归评估：**从当前棋局出发，递归地模拟游戏的未来状态。每一个叶子节点都会根据游戏规则评估其得分，通常通过一个启发式的评估函数来表示每个局面（例如，若AI即将获胜，得分较高；若对手即将获胜，得分较低）。

**极大值与极小值：**在递归的过程中，极大值玩家（AI）选择能够让其得分最大的步骤，极小值玩家（对手）选择能够使得AI得分最小的步骤。

**回溯：**一旦到达叶子节点，回溯（Backpropagation）会根据每一层的极大极小规则选择最优解。极大值玩家会选择具有最高得分的路径，极小值玩家则选择得分最低的路径。

## Alpha-beta剪枝

通过对上述极大极小值搜索的介绍，不难发现它所需的计算量是十分庞大的，并且随着搜索层数（深度）的增加呈几何式增长。为此，我们需要优化算法以减少搜索的次数。可以发现，并不是所有的分支都需要被探索。例如，如果某个节点的评估值已经足够好，后续的节点将不会影响这个节点的决策，这时就可以提前停止对这些节点的计算。为此，我们可以在极大极小值搜索的基础上使用  $\alpha - \beta$  剪枝。

**$\alpha$  (Alpha)：**当前极大值玩家可以确保的最佳得分，即在当前层之前找到的最大得分。

**$\beta$  (Beta)：**当前极小值玩家可以确保的最佳得分，即在当前层之前找到的最小得分。

当算法在搜索树中遍历时，极大值玩家会试图最大化得分，而极小值玩家则会试图最小化得分。在递归的过程中，如果发现某个节点的评估值已经不可能影响父节点的选择，就可以直接剪掉该节点的所有子节点。如果一个极小值节点的值小于或等于其父节点的  $\alpha$  值，或者如果一个极大值节点的值大于或等于其父节点的  $\beta$  值，那么就可以停止对当前节点及其子树的进一步探索，因为这些节点的评估不会影响最终决策。

以下是一个  $\alpha - \beta$  剪枝的具体流程：

假设我们正在使用极大极小值算法来搜索一个五子棋的得分树：

对于极大值玩家， $\alpha$  是当前已经找到的最大得分， $\beta$  是极小值玩家可以接受的最小得分。

对于极小值玩家， $\alpha$  是极大值玩家可以接受的最大得分， $\beta$  是当前已经找到的最小得分。

**搜索过程中的剪枝规则：**

**对于极大值玩家：**

- 如果某个子节点的评估值大于等于  $\beta$ （即当前极小值玩家已经有机会阻止极大值玩家的得分），则可以停止对该子节点的进一步探索。
- 否则，将当前的  $\alpha$  更新为更大的值。

**对于极小值玩家：**

- 如果某个子节点的评估值小于等于  $\alpha$ （即当前极大值玩家已经有机会得到更好的得分），则可以停止对该子节点的进一步探索。
- 否则，将当前的  $\beta$  更新为更小的值。

```
vector<maxScorePoint> MsgProcess::getEmpty()
{
    vector<maxScorePoint> empty;
    for (int r = 1; r < 14; r++)
    {
        for (int c = 1; c < 14; c++)
        {
            if (cell[r][c] == 0)
            {
                if (cell[r][c + 1] != 0)
                {
                    empty.push_back({ r, c });
                    continue;
                }
                if (cell[r][c - 1] != 0)
                {
                    empty.push_back({ r, c });
                    continue;
                }
                if (cell[r + 1][c] != 0)
                {
                    empty.push_back({ r, c });
                    continue;
                }
            }
        }
    }
}
```

```
        if (cell[r - 1][c] != 0)
        {
            empty.push_back({ r, c });
            continue;
        }
        if (cell[r + 1][c + 1] != 0)
        {
            empty.push_back({ r, c });
            continue;
        }
        if (cell[r - 1][c + 1] != 0)
        {
            empty.push_back({ r, c });
            continue;
        }
        if (cell[r + 1][c - 1] != 0)
        {
            empty.push_back({ r, c });
            continue;
        }
        if (cell[r - 1][c - 1] != 0)
        {
            empty.push_back({ r, c });
            continue;
        }
    }
}

for (int r = 0; r < 15; r++)
{
    if (r < 14)
    {
        if (cell[r][0] == 0)
        {
            if (cell[r + 1][0] != 0)
            {
                empty.push_back({ r, 0 });
                continue;
            }
            if (cell[r + 1][1] != 0)
            {
                empty.push_back({ r, 0 });
                continue;
            }
        }
        if (cell[r][14] == 0)
        {
            if (cell[r + 1][14] != 0)
            {
                empty.push_back({ r, 14 });
                continue;
            }
            if (cell[r + 1][13] != 0)
            {
                empty.push_back({ r, 14 });
                continue;
            }
        }
    }
}
```

```
        }
    }
}
if (r > 0)
{
    if (cell[r][0] == 0)
    {
        if (cell[r - 1][0] != 0)
        {
            empty.push_back({ r, 0 });
            continue;
        }
        if (cell[r - 1][1] != 0)
        {
            empty.push_back({ r, 0 });
            continue;
        }
    }
    if (cell[r][14] == 0)
    {
        if (cell[r - 1][14] != 0)
        {
            empty.push_back({ r, 14 });
            continue;
        }
        if (cell[r - 1][13] != 0)
        {
            empty.push_back({ r, 14 });
            continue;
        }
    }
}
if (cell[r][0] == 0 && cell[r][1] != 0)
{
    empty.push_back({ r, 0 });
    continue;
}
if (cell[r][14] == 0 && cell[r][13] != 0)
{
    empty.push_back({ r, 14 });
    continue;
}
}
for (int c = 1; c < 14; c++)
{
    if (cell[0][c] == 0)
    {
        if (cell[0][c - 1] != 0)
        {
            empty.push_back({ 0, c });
            continue;
        }
        if (cell[0][c + 1] != 0)
        {
            empty.push_back({ 0, c });
            continue;
        }
    }
}
```

```
        }
        if (cell[1][c - 1] != 0)
        {
            empty.push_back({ 0, c });
            continue;
        }
        if (cell[1][c + 1] != 0)
        {
            empty.push_back({ 0, c });
            continue;
        }
        if (cell[1][c] != 0)
        {
            empty.push_back({ 0, c });
            continue;
        }
    }
    if (cell[14][c] == 0)
    {
        if (cell[14][c - 1] != 0)
        {
            empty.push_back({ 14, c });
            continue;
        }
        if (cell[14][c + 1] != 0)
        {
            empty.push_back({ 14, c });
            continue;
        }
        if (cell[13][c - 1] != 0)
        {
            empty.push_back({ 14, c });
            continue;
        }
        if (cell[13][c + 1] != 0)
        {
            empty.push_back({ 14, c });
            continue;
        }
        if (cell[13][c] != 0)
        {
            empty.push_back({ 14, c });
            continue;
        }
    }
}

return empty;
}

vector<maxScorePoint> MsgProcess::inspire(int Color)
{
    vector<maxScorePoint> p = getEmpty();
    for (int i = 0; i < p.size(); i++)
    {
        for (int j = 1; j < p.size() - i; j++)
        {
```

```

        if (evaluateValue(p[j - 1].row, p[j - 1].col, Color) > evaluateValue(p[j].row,
p[j].col, Color))
        {
            int temp1 = p[j].row;
            int temp2 = p[j].col;
            p[j].row = p[j - 1].row;
            p[j].col = p[j - 1].col;
            p[j - 1].row = temp1;
            p[j - 1].col = temp2;
        }
    }
    return p;
}

maxScorePoint MsgProcess::getMinMaxEvaluate(int depth, int aiColor) {
    vector<maxScorePoint> maxPoint; //Store the points with the same score.
    int alpha = INT_MIN; //Initialize maxValue
    int beta = INT_MAX;
    int k = 0; //Store the number of the points that have the same score.
    /*Scan the whole board. */
    vector<maxScorePoint> po = inspire(aiColor);
    for (int i = po.size()-1; i >=0; i--)
    {
        int r = po[i].row;
        int c = po[i].col;

        cell[r][c] = aiColor; //if we put the AI chess at [i][j] ChessBox[][]=AIcolor
        if (evaluate_value(aiColor) >= 50000)
        {
            return { r, c };
        }

        int temp = Min_Max_Value(depth - 1, alpha, beta, 1, aiColor); //temp to store the min
value

        /* Compare temp and maxValue, update maxValue, update maxPoint and k */
        if (temp > alpha) {
            alpha = temp;
            maxPoint.clear();
            maxPoint.push_back({ r, c });
        }

        else if (temp == alpha)
        {
            maxPoint.push_back({ r, c });
        }
        cell[r][c] = 0;
    }

    int n = 0;
    if (maxPoint.size() > 1)
    {
        int m = scoreMap[maxPoint[0].row][maxPoint[0].col];
        for (int i = 1; i < maxPoint.size(); i++)
        {
            int temp = scoreMap[maxPoint[i].row][maxPoint[i].col];
            if (temp > m)

```



```
        {  
            m = temp;  
            n = i;  
        }  
    }  
}  
return maxPoint[n];  
}
```

由此，我们可以得到一个实现AI下五子棋的简单流程

